

Python & Scikit-learn

Interview Preparation Notes — Strings, Exceptions & the ML Workflow

1. Python — String Manipulation

String practice for ML / AI / Data Engineering work shouldn't stop at basic methods — these come up constantly in data cleaning, preprocessing, parsing, log processing, API responses, and file handling.

Most Important Methods

Method	Use case
<code>lower()</code>	Normalize text before comparison/search
<code>upper()</code>	Standardize categorical/text values
<code>title()</code>	Convert names/titles to proper case
<code>strip()</code>	Remove unwanted spaces/newlines from input
<code>lstrip()</code>	Remove leading whitespace
<code>rstrip()</code>	Remove trailing whitespace
<code>split()</code>	Convert text into tokens/list
<code>join()</code>	Combine list of strings into one string
<code>replace()</code>	Clean or replace unwanted text
<code>count()</code>	Count occurrences of a substring
<code>find()</code>	Find position of substring
<code>index()</code>	Find position; raises error if not found
<code>startswith()</code>	Validate prefixes
<code>endswith()</code>	Validate suffixes
<code>in</code>	Check whether text exists
<code>len()</code>	Get string length

Examples

```
text = " Python is AMAZING "  
  
text.strip()  
text.lower()  
text.upper()  
text.title()
```

split() — string → list

Very important for data parsing.

```
text = "Python,SQL,Pandas"  
  
skills = text.split(",")
```

Result:

```
['Python', 'SQL', 'Pandas']
```

join() — reverse operation of split()

```
skills = ['Python', 'SQL', 'Pandas']  
  
result = " | ".join(skills)
```

Result:

```
Python | SQL | Pandas
```

■ Remember

```
split() → string → list  
join() → list → string
```

replace() — cleaning data

```
text = "Python Developer"  
  
text.replace("Developer", "Engineer")
```

find() — locate a substring

```
text = "machine learning"  
  
print(text.find("learning"))
```

Returns the starting index — useful when parsing text or checking where something occurs.

startswith() / endswith() — validating filenames

```
filename = "data.csv"

filename.endswith(".csv")
```

Very common when processing files.

Other Useful String Methods

- `isalnum()`
- `isalpha()`
- `isdigit()`
- `isnumeric()`
- `isspace()`
- `islower()`
- `isupper()`

Useful when validating and cleaning raw data.

```
age = "25"

age.isdigit()
# True
```

Practical Pattern — Processing Employee Data

```
employees = [
    "Alice | Python,SQL,Pandas",
    " Bob | Python,NumPy,Scikit-learn ",
    "CHARLIE | SQL,Pandas,Power BI"
]

for employee in employees:
    name, skills = employee.split("|")

    name = name.strip().title()
    skills = skills.strip().split(",")

    print(f"{name} -> {skills} -> {len(skills)} skills")
```

This is much closer to the kind of Python used in real data-processing work than isolated method calls.

2. Python — Exception Handling

try / except

```
try:
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Use when an operation can fail and you want the program to handle the error gracefully.

finally

```
try:
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Execution completed")
```

finally executes whether an exception occurs or not — useful for cleanup operations, such as closing resources.

raise

Used when you want to deliberately generate an exception.

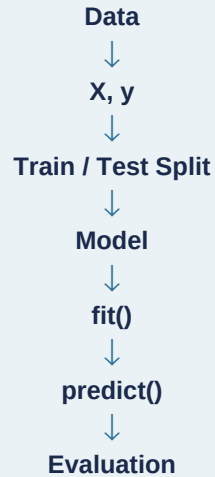
```
def check_salary(salary):
    if salary < 0:
        raise ValueError("Salary cannot be negative")
    print("Valid salary")
```

Common pattern:

```
if invalid:
    raise ValueError("message")
```

3. Scikit-learn — Complete ML Workflow

The basic classification workflow, practiced end to end:



Dataset

A DataFrame containing the following columns:

- age
- salary
- experience
- purchased (target)

X and y

```
X = df[["age", "salary", "experience"]]
y = df["purchased"]
```

■ Remember

X → features / input
y → target / output

Train / Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Why?

- Training data → learn patterns
- Testing data → evaluate performance on unseen data

Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
```

Train:

```
model.fit(X_train, y_train)
```

Predict:

```
y_pred = model.predict(X_test)
```

■ Remember

`fit()` → learn

`predict()` → predict

Evaluation

Accuracy

```
from sklearn.metrics import accuracy_score

accuracy_score(y_test, y_pred)
```

Measures the proportion of predictions that are correct.

Confusion Matrix

```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, y_pred)
```

Helps understand:

- True Positive
- True Negative
- False Positive
- False Negative

Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Provides metrics such as:

- Precision
- Recall
- F1-score
- Support

New Data Prediction

For a new customer:

```
new_customer = pd.DataFrame(
    [[33, 58000, 7]],
    columns=["age", "salary", "experience"]
)

model.predict(new_customer)
```

Important Issue Encountered

The model produced a feature scaling / convergence warning because the feature magnitudes were mismatched:

- age → small values
- experience → small values
- salary → large values

This leads directly into the next Scikit-learn topic: **feature scaling**.

■ **Next Scikit-learn Topic:** Feature Scaling (StandardScaler / MinMaxScaler)